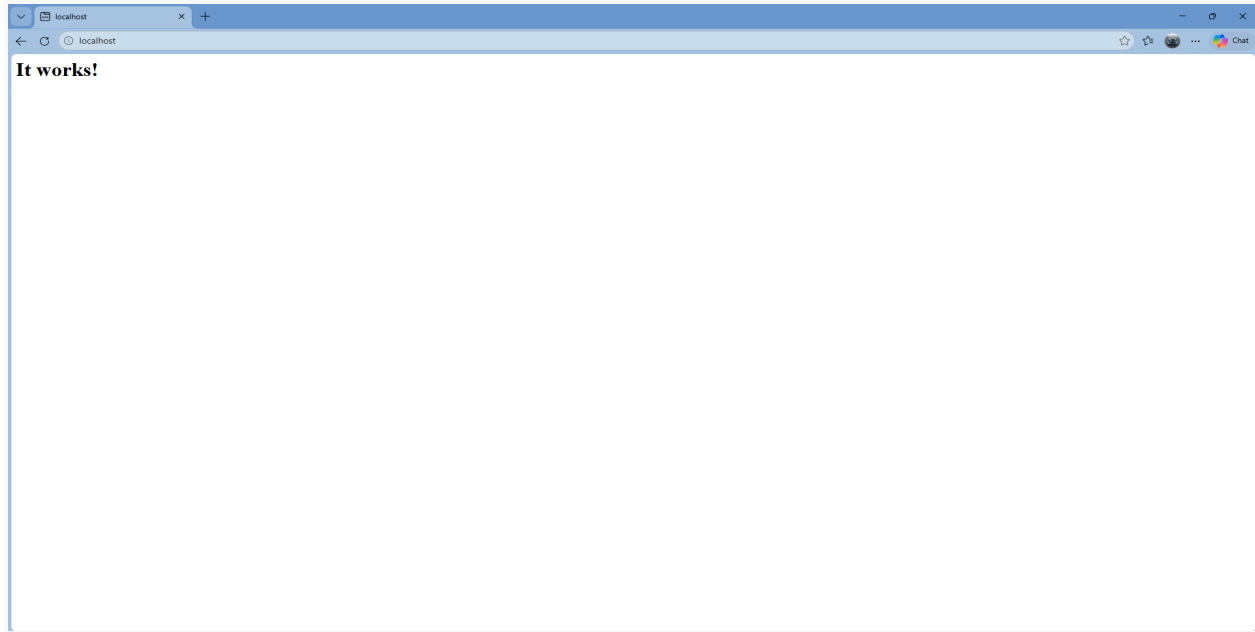


L2T11 Task: PKI and Man-in-the-Middle Attacks

Task 2:



Task 3:

Microsoft Windows [Version 10.0.26200.8457]

(c) Microsoft Corporation. All rights reserved.

```
C:\Users\Splen>cd C:\Apache24\conf
```

```
C:\Apache24\conf>openssl x509 -in server.crt -text -noout
```

Certificate:

Data:

Version: 3 (0x2)

Serial Number:

5e:27:3d:57:31:ca:36:4c:8b:ec:7d:8b:88:dc:80:4d:10:6a:50:5f

Signature Algorithm: sha256WithRSAEncryption

Issuer: CN=localhost

Validity

Not Before: May 27 08:02:18 2026 GMT

Not After : May 27 08:02:18 2027 GMT

Subject: CN=localhost

Subject Public Key Info:

Public Key Algorithm: rsaEncryption

Public-Key: (2048 bit)

Modulus:

00:b3:a7:a7:85:83:fd:2c:b9:14:ab:92:f0:95:02:
da:9b:1a:b1:67:d5:2d:60:03:8e:05:55:5b:ef:9c:
6d:93:e0:97:c3:50:85:fc:2c:3e:b2:ed:10:32:ae:
3d:87:60:fc:c3:19:d4:46:6b:d7:1e:a6:30:7a:91:
55:6f:f0:35:bb:72:d3:a3:7e:08:e3:36:91:65:16:
46:82:1d:f3:a8:de:1f:b7:87:2d:92:15:59:d4:b7:
3e:7e:2c:cd:37:a1:cc:66:80:74:80:bf:66:be:ad:
33:bf:f8:bf:99:7b:5b:65:4e:d3:19:fe:b7:94:4b:
d7:cb:95:62:00:b7:d4:fb:05:ba:5d:4e:42:83:b9:
e1:75:6e:bf:7b:b0:0c:a4:b3:02:a6:5f:9a:21:9b:
f0:da:e1:a2:81:f9:6e:de:92:3e:62:24:b0:2c:b1:
5f:3b:ff:24:3a:f7:d4:6e:32:ed:d6:af:b8:d7:ca:
98:e8:29:0c:95:3e:8f:2c:34:f5:c0:6e:38:3a:18:
64:40:50:b9:de:e2:dc:14:be:5b:18:9a:0e:f5:5e:
7f:0b:fb:1f:3d:38:f0:d9:f2:d4:0a:d9:6a:c1:46:
1b:80:e7:a7:7a:49:55:75:ac:3b:bf:81:95:a9:98:
5a:30:d8:f9:8f:29:78:12:5b:3b:74:82:59:6f:c9:
d2:05

Exponent: 65537 (0x10001)

X509v3 extensions:

X509v3 Subject Alternative Name:

DNS:localhost

X509v3 Subject Key Identifier:

72:5C:2D:78:D6:30:49:30:CD:48:95:D9:36:C7:88:04:C0:0E:A1:DB

Signature Algorithm: sha256WithRSAEncryption

Signature Value:

55:46:2e:d3:db:4c:31:d2:b8:da:3b:2b:be:21:95:f9:d4:71:
c1:93:f8:bf:15:86:d1:2c:4e:7c:eb:b8:83:ab:0a:9e:14:e9:
66:59:ed:13:8c:10:4d:7e:07:ab:aa:5f:dd:3e:b7:60:b3:36:
5f:0c:6d:fa:b8:6f:49:5d:11:93:3e:a1:22:04:7f:cc:a5:e7:
62:ec:88:30:59:f2:09:15:c2:22:11:a1:cd:6f:4c:56:ee:3e:
6c:3a:d5:1c:8d:95:a8:cf:80:6e:e7:3b:64:e8:f2:42:01:c3:
ee:92:c3:f7:a2:5e:ae:0c:e0:1e:f7:62:d6:a5:30:b6:dd:5e:
70:d0:e5:97:e7:f1:92:b8:c4:8a:b9:7d:dd:03:d3:36:56:5e:
ee:d3:9a:48:9f:19:2a:09:79:ac:23:35:21:58:d6:14:03:9c:
e8:80:c1:52:ad:46:d5:e7:29:4d:38:0d:97:d3:57:f8:46:af:
a9:59:1c:6d:cd:1e:79:03:b2:1f:5e:83:41:86:4d:79:df:b9:
d9:b2:71:ac:db:e4:e4:7a:42:6e:88:4a:47:ea:30:14:64:a3:
46:ca:28:b5:b5:51:ad:6c:5f:6b:65:34:12:40:09:3f:2f:59:
33:05:88:64:77:49:05:f5:f9:31:84:9e:01:4c:19:e0:77:fc:
e0:84:68:3e

C:\Apache24\conf>

Task 4:

Microsoft Windows [Version 10.0.26200.8524]

(c) Microsoft Corporation. All rights reserved.

```
C:\Users\Splen>type C:\Apache24\conf\extra\httpd-ssl.conf
```

```
#
```

```
# This is the Apache server configuration file providing SSL support.
```

```
# It contains the configuration directives to instruct the server how to
```

```
# serve pages over an https connection. For detailed information about these
```

```
# directives see <URL:http://httpd.apache.org/docs/2.4/mod/mod_ssl.html>
```

```
#
```

```
# Do NOT simply read the instructions in here without understanding
```

```
# what they do. They're here only as hints or reminders. If you are unsure
```

```
# consult the online docs. You have been warned.
```

```
#
```

```
# Required modules: mod_log_config, mod_setenvif, mod_ssl,
```

```
# socache_shmcb_module (for default value of SSLSessionCache)
```

```
#
```

```
# Pseudo Random Number Generator (PRNG):
```

```
# Configure one or more sources to seed the PRNG of the SSL library.
```

```
# The seed data should be of good random quality.
```

```
# WARNING! On some platforms /dev/random blocks if not enough entropy
```

```
# is available. This means you then cannot use the /dev/random device
```

```
# because it would lead to very long connection times (as long as
```

```
# it requires to make more entropy available). But usually those
```

```
# platforms additionally provide a /dev/urandom device which doesn't
```

```
# block. So, if available, use this one instead. Read the mod_ssl User
```

```
# Manual for more details.
```

```
#
```

```
#SSLRandomSeed startup file:/dev/random 512
```

```
#SSLRandomSeed startup file:/dev/urandom 512
```

```
#SSLRandomSeed connect file:/dev/random 512
```

```
#SSLRandomSeed connect file:/dev/urandom 512
```

```
#
```

```
# When we also provide SSL we have to listen to the
```

```
# standard HTTP port (see above) and to the HTTPS port
```

```
#
```

```
Listen 443
```

```
##
```

SSL Global Context

##

All SSL configuration in this context applies both to

the main server and all SSL-enabled virtual hosts.

##

SSL Cipher Suite:

List the ciphers that the client is permitted to negotiate,

and that httpd will negotiate as the client of a proxied server.

See the OpenSSL documentation for a complete list of ciphers, and

ensure these follow appropriate best practices for this deployment.

httpd 2.2.30, 2.4.13 and later force-disable aNULL, eNULL and EXP ciphers,

while OpenSSL disabled these by default in 0.9.8zf/1.0.0r/1.0.1m/1.0.2a.

SSLCipherSuite HIGH:MEDIUM:!MD5:!RC4:!3DES

SSLProxyCipherSuite HIGH:MEDIUM:!MD5:!RC4:!3DES

By the end of 2016, only TLSv1.2 ciphers should remain in use.

Older ciphers should be disallowed as soon as possible, while the

kRSA ciphers do not offer forward secrecy. These changes inhibit

older clients (such as IE6 SP2 or IE8 on Windows XP, or other legacy

non-browser tooling) from successfully connecting.

#

To restrict mod_ssl to use only TLSv1.2 ciphers, and disable

those protocols which do not support forward secrecy, replace

the SSLCipherSuite and SSLProxyCipherSuite directives above with

the following two directives, as soon as practical.

SSLCipherSuite HIGH:MEDIUM:!SSLv3:!kRSA

SSLProxyCipherSuite HIGH:MEDIUM:!SSLv3:!kRSA

User agents such as web browsers are not configured for the user's

own preference of either security or performance, therefore this

must be the prerogative of the web server administrator who manages

cpu load versus confidentiality, so enforce the server's cipher order.

SSLHonorCipherOrder on

SSL Protocol support:

List the protocol versions which clients are allowed to connect with.

Disable SSLv3 by default (cf. RFC 7525 3.1.1). TLSv1 (1.0) should be

disabled as quickly as practical. By the end of 2016, only the TLSv1.2

protocol or later should remain in use.

SSLProtocol all -SSLv3

SSLProxyProtocol all -SSLv3

Pass Phrase Dialog:

```
# Configure the pass phrase gathering process.
# The filtering dialog program ('builtin' is an internal
# terminal dialog) has to provide the pass phrase on stdout.
SSLPassPhraseDialog builtin

# Inter-Process Session Cache:
# Configure the SSL Session Cache: First the mechanism
# to use and second the expiring timeout (in seconds).
#SSLSessionCache      "dbm:${SRVROOT}/logs/ssl_scache"
#SSLSessionCache      "shmcb:${SRVROOT}/logs/ssl_scache(512000)"
SSLSessionCacheTimeout 300

# OCSP Stapling (requires OpenSSL 0.9.8h or later)
#
# This feature is disabled by default and requires at least
# the two directives SSLUseStapling and SSLStaplingCache.
# Refer to the documentation on OCSP Stapling in the SSL/TLS
# How-To for more information.
#
# Enable stapling for all SSL-enabled servers:
#SSLUseStapling On

# Define a relatively small cache for OCSP Stapling using
# the same mechanism that is used for the SSL session cache
# above. If stapling is used with more than a few certificates,
# the size may need to be increased. (AH01929 will be logged.)
#SSLStaplingCache "shmcb:${SRVROOT}/logs/ssl_stapling(32768)"

# Seconds before valid OCSP responses are expired from the cache
#SSLStaplingStandardCacheTimeout 3600

# Seconds before invalid OCSP responses are expired from the cache
#SSLStaplingErrorCacheTimeout 600

##
## SSL Virtual Host Context
##

<VirtualHost _default_:443>

# General setup for the virtual host
DocumentRoot "${SRVROOT}/htdocs"
ServerName localhost:443
ServerAdmin you@example.com
```

```
ErrorLog "${SRVROOT}/logs/error_log"
TransferLog "${SRVROOT}/logs/access_log"
```

```
# SSL Engine Switch:
# Enable/Disable SSL for this virtual host.
SSLEngine on
```

```
# Server Certificate:
# Point SSLCertificateFile at a PEM encoded certificate. If
# the certificate is encrypted, then you will be prompted for a
# pass phrase. Note that a kill -HUP will prompt again. Keep
# in mind that if you have both an RSA and a DSA certificate you
# can configure both in parallel (to also allow the use of DSA
# ciphers, etc.)
# Some ECC cipher suites (http://www.ietf.org/rfc/rfc4492.txt)
# require an ECC certificate which can also be configured in
# parallel.
SSLCertificateFile "C:/Apache24/conf/server.crt"
#SSLCertificateFile "${SRVROOT}/conf/server-dsa.crt"
#SSLCertificateFile "${SRVROOT}/conf/server-ecc.crt"
```

```
# Server Private Key:
# If the key is not combined with the certificate, use this
# directive to point at the key file. Keep in mind that if
# you've both a RSA and a DSA private key you can configure
# both in parallel (to also allow the use of DSA ciphers, etc.)
# ECC keys, when in use, can also be configured in parallel
SSLCertificateKeyFile "C:/Apache24/conf/server.key"
#SSLCertificateKeyFile "${SRVROOT}/conf/server-dsa.key"
#SSLCertificateKeyFile "${SRVROOT}/conf/server-ecc.key"
```

```
# Server Certificate Chain:
# Point SSLCertificateChainFile at a file containing the
# concatenation of PEM encoded CA certificates which form the
# certificate chain for the server certificate. Alternatively
# the referenced file can be the same as SSLCertificateFile
# when the CA certificates are directly appended to the server
# certificate for convenience.
#SSLCertificateChainFile "${SRVROOT}/conf/server-ca.crt"
```

```
# Certificate Authority (CA):
# Set the CA certificate verification path where to find CA
# certificates for client authentication or alternatively one
# huge file containing all of them (file must be PEM encoded)
```

```
# Note: Inside SSLCACertificatePath you need hash symlinks
#   to point to the certificate files. Use the provided
#   Makefile to update the hash symlinks after changes.
#SSLCACertificatePath "${SRVROOT}/conf/ssl.crt"
#SSLCACertificateFile "${SRVROOT}/conf/ssl.crt/ca-bundle.crt"
```

```
# Certificate Revocation Lists (CRL):
# Set the CA revocation path where to find CA CRLs for client
# authentication or alternatively one huge file containing all
# of them (file must be PEM encoded).
# The CRL checking mode needs to be configured explicitly
# through SSLCARevocationCheck (defaults to "none" otherwise).
# Note: Inside SSLCARevocationPath you need hash symlinks
#   to point to the certificate files. Use the provided
#   Makefile to update the hash symlinks after changes.
#SSLCARevocationPath "${SRVROOT}/conf/ssl.crl"
#SSLCARevocationFile "${SRVROOT}/conf/ssl.crl/ca-bundle.crl"
#SSLCARevocationCheck chain
```

```
# Client Authentication (Type):
# Client certificate verification type and depth. Types are
# none, optional, require and optional_no_ca. Depth is a
# number which specifies how deeply to verify the certificate
# issuer chain before deciding the certificate is not valid.
#SSLVerifyClient require
#SSLVerifyDepth 10
```

```
# TLS-SRP mutual authentication:
# Enable TLS-SRP and set the path to the OpenSSL SRP verifier
# file (containing login information for SRP user accounts).
# Requires OpenSSL 1.0.1 or newer. See the mod_ssl FAQ for
# detailed instructions on creating this file. Example:
# "openssl srp -srpvfile ${SRVROOT}/conf/passwd.srpv -add username"
#SSLSRPVerifierFile "${SRVROOT}/conf/passwd.srpv"
```

```
# Access Control:
# With SSLRequire you can do per-directory access control based
# on arbitrary complex boolean expressions containing server
# variable checks and other lookup directives. The syntax is a
# mixture between C and Perl. See the mod_ssl documentation
# for more details.
#<Location />
#SSLRequire ( %{SSL_CIPHER} !~ m/^(EXP|NULL)/ \
#   and %{SSL_CLIENT_S_DN_O} eq "Snake Oil, Ltd." \
```

```

# and %{SSL_CLIENT_S_DN_OU} in {"Staff", "CA", "Dev"} \
# and %{TIME_WDAY} >= 1 and %{TIME_WDAY} <= 5 \
# and %{TIME_HOUR} >= 8 and %{TIME_HOUR} <= 20 ) \
# or %{REMOTE_ADDR} =~ m/^192\.76\.162\.[0-9]+\$/
#</Location>

# SSL Engine Options:
# Set various options for the SSL engine.
# o FakeBasicAuth:
# Translate the client X.509 into a Basic Authorisation. This means that
# the standard Auth/DBMAuth methods can be used for access control. The
# user name is the `one line' version of the client's X.509 certificate.
# Note that no password is obtained from the user. Every entry in the user
# file needs this password: `xxj31ZMTZzkVA'.
# o ExportCertData:
# This exports two additional environment variables: SSL_CLIENT_CERT and
# SSL_SERVER_CERT. These contain the PEM-encoded certificates of the
# server (always existing) and the client (only existing when client
# authentication is used). This can be used to import the certificates
# into CGI scripts.
# o StdEnvVars:
# This exports the standard SSL/TLS related `SSL_*' environment variables.
# Per default this exportation is switched off for performance reasons,
# because the extraction step is an expensive operation and is usually
# useless for serving static content. So one usually enables the
# exportation for CGI and SSI requests only.
# o StrictRequire:
# This denies access when "SSLRequireSSL" or "SSLRequire" applied even
# under a "Satisfy any" situation, i.e. when it applies access is denied
# and no other module can change it.
# o OptRenegotiate:
# This enables optimized SSL connection renegotiation handling when SSL
# directives are used in per-directory context.
#SSLOptions +FakeBasicAuth +ExportCertData +StrictRequire
<FilesMatch "\.(cgi|shtml|phtml|php)$">
    SSLOptions +StdEnvVars
</FilesMatch>
<Directory "${SRVROOT}/cgi-bin">
    SSLOptions +StdEnvVars
</Directory>

# SSL Protocol Adjustments:
# The safe and default but still SSL/TLS standard compliant shutdown
# approach is that mod_ssl sends the close notify alert but doesn't wait for

```



```

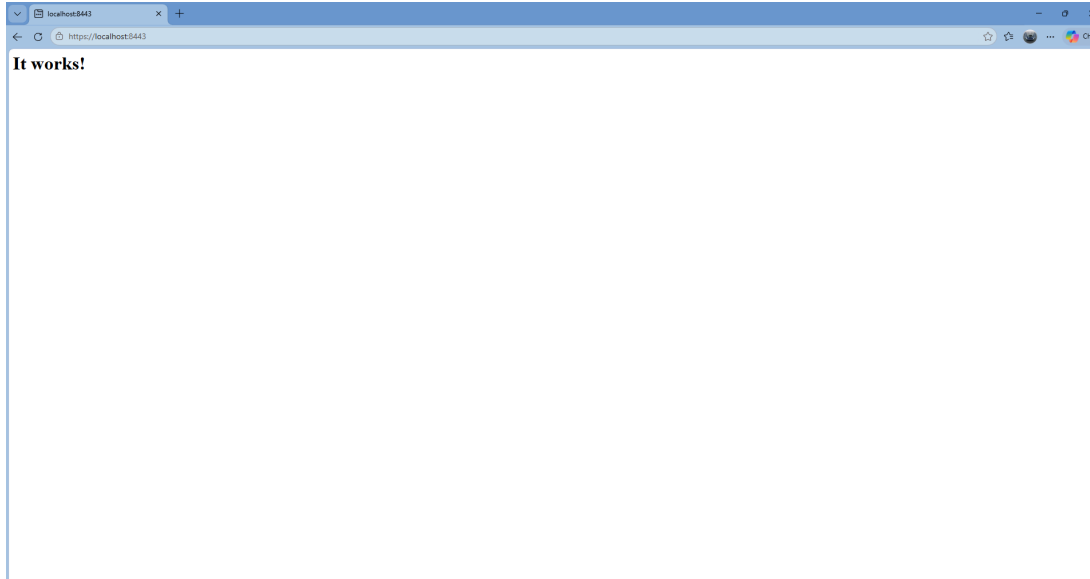
# the close notify alert from client. When you need a different shutdown
# approach you can use one of the following variables:
# o ssl-unclean-shutdown:
#   This forces an unclean shutdown when the connection is closed, i.e. no
#   SSL close notify alert is sent or allowed to be received. This violates
#   the SSL/TLS standard but is needed for some brain-dead browsers. Use
#   this when you receive I/O errors because of the standard approach where
#   mod_ssl sends the close notify alert.
# o ssl-accurate-shutdown:
#   This forces an accurate shutdown when the connection is closed, i.e. a
#   SSL close notify alert is send and mod_ssl waits for the close notify
#   alert of the client. This is 100% SSL/TLS standard compliant, but in
#   practice often causes hanging connections with brain-dead browsers. Use
#   this only for browsers where you know that their SSL implementation
#   works correctly.
# Notice: Most problems of broken clients are also related to the HTTP
# keep-alive facility, so you usually additionally want to disable
# keep-alive for those clients, too. Use variable "nokeepalive" for this.
# Similarly, one has to force some clients to use HTTP/1.0 to workaround
# their broken HTTP/1.1 implementation. Use variables "downgrade-1.0" and
# "force-response-1.0" for this.
BrowserMatch "MSIE [2-5]" \
    nokeepalive ssl-unclean-shutdown \
    downgrade-1.0 force-response-1.0

# Per-Server Logging:
# The home of a custom SSL log file. Use this when you want a
# compact non-error SSL logfile on a virtual host basis.
CustomLog "${SRVROOT}/logs/ssl_request_log" \
    "%t %h %{SSL_PROTOCOL}x %{SSL_CIPHER}x \"%r\" %b"

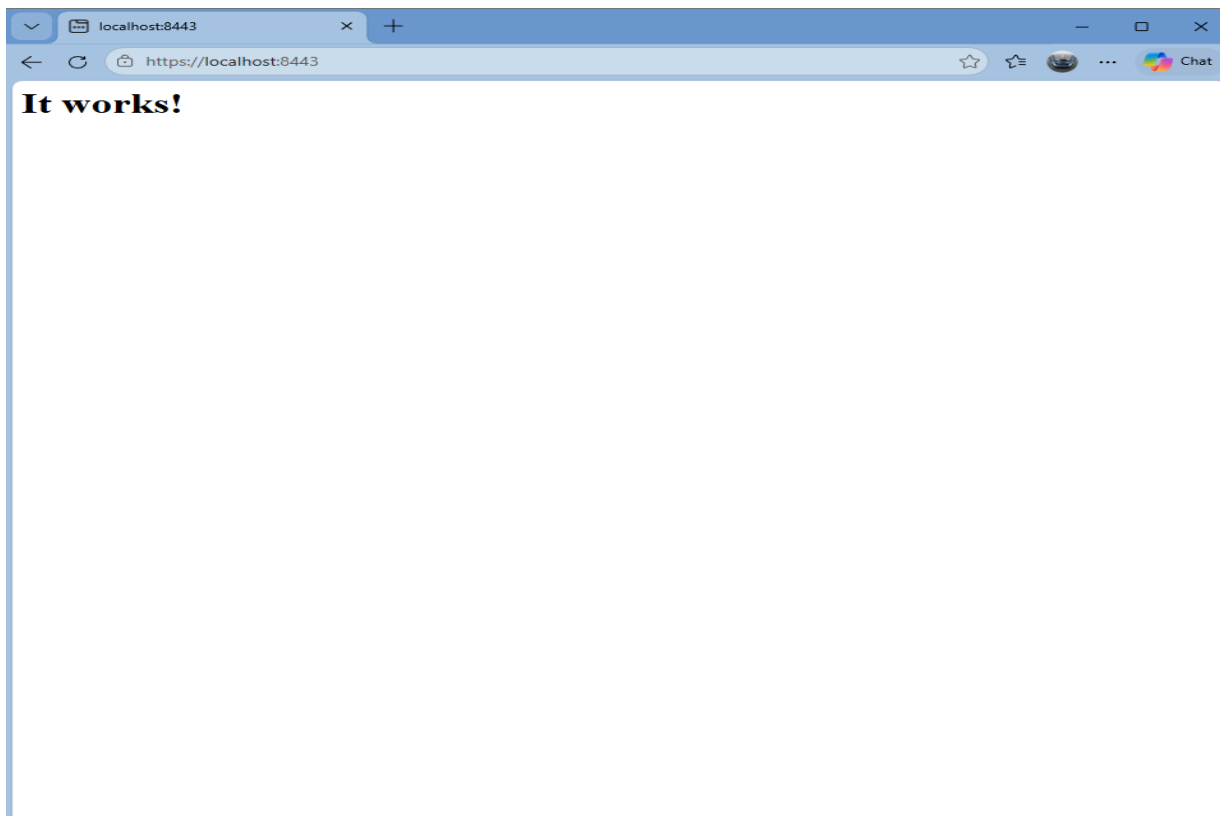
</VirtualHost>

```

Task 5:



Task 6:



Task 7:

The certificate is issued by: CN=localhost
The certificate expires: 27 May 2027

